课程笔记

CS336 第 9 讲：Scaling Laws I

Codex 基于 Stanford CS336 2025 第 9 讲视频、字幕与课程材料整理

2026 年 4 月 7 日

视频作者/频道：Stanford Online
发布日期：2025 年 05 月 15 日
视频时长：1:05:18
视频链接：`https://www.youtube.com/watch?v=6Q-ESEmDf4Q`

目录

1 为什么 scaling laws 是大模型时代的预算计算器 **2**
1.1 从“一万个 H100”开始，课程到底在问什么 2
1.2 历史背景：scaling 不是 LLM 时代突然发明的 2
1.3 本章小结 3

2 数据 scaling law：为什么 log-log 图会变直 **3**
2.1 什么叫 data scaling law 3
2.2 均值估计这个玩具例子为什么很关键 4
2.3 为什么神经网络里的 exponent 仍然是个谜 5
2.4 本章小结 6

3 数据不只有规模，还有组成、重复和分布偏移 **6**
3.1 数据组成往往改的是 offset，而不一定改 slope 6
3.2 重复数据为什么不能按名义 token 数来算 6
3.3 本章小结 6

4 模型工程也可以做 scaling analysis **6**
4.1 除了数据量，还能外推什么 6
4.2 critical batch size 说明 batch 不是越大越好 7
4.3 muP 和 scale-aware 学习率只是这一讲的预告 7
4.4 本章小结 7

5 联合数据-模型 scaling：回答“该加数据还是加模型” **7**
5.1 真正的训练规划一定是多轴联动的 7
5.2 为什么小模型会浪费大数据 7
5.3 本章小结 8

6 总结与延伸 **8**

1 为什么 scaling laws 是大模型时代的预算计算器

1.1 从“一万个 H100”开始，课程到底在问什么

第 9 讲开场故意把问题推到一个非常工程化的极端：如果有人给你一万个 H100 和一个月时间，让你做一个强开源语言模型，你会怎样分配这笔预算？这不是在问你会不会搭集群，也不是在问你会不会清洗数据，而是在问一个更根本的问题：**在算力和时间固定时，模型大小、训练 token 数和超参数应该怎样联动选择**。

课程想用 scaling laws 回答的正是这个问题。旧范式是直接在大模型上试深度、宽度、优化器、学习率和 batch，代价极高，反馈又慢；新范式则是在较小模型上系统扫点，拟合出相对稳定的规律，再把这些规律外推到大模型区间。也就是说，scaling laws 的价值首先不是“解释世界”，而是“替工程决策降本”。

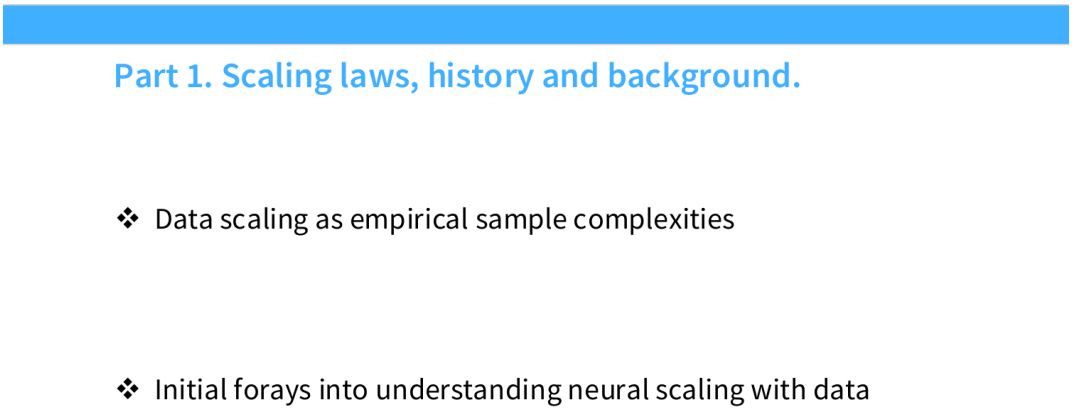


图 1: 课程从“给你大规模算力，你究竟该训练怎样的模型”切入 scaling laws 的工程意义。

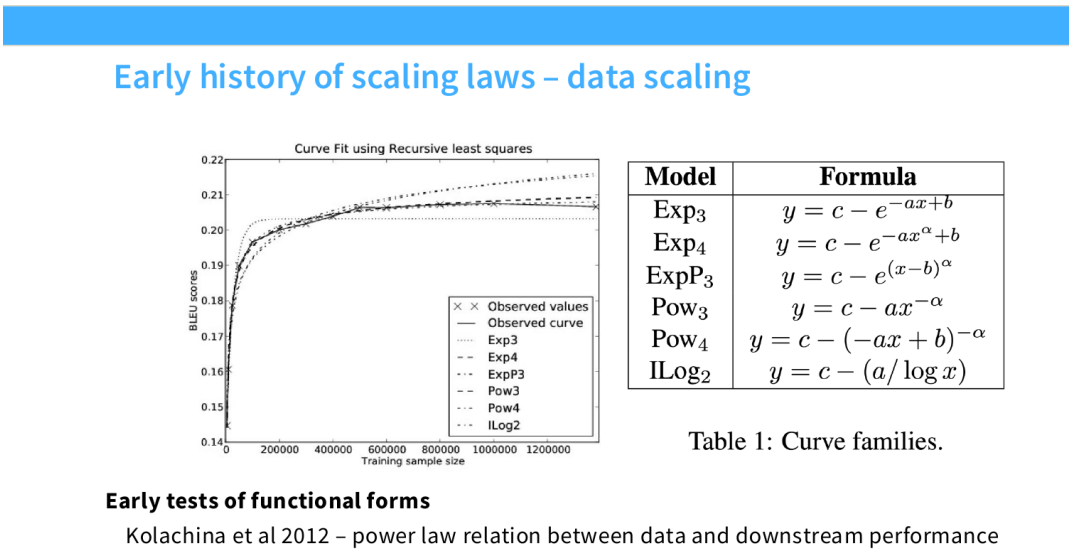
这一讲的压缩目标

把“训练大模型”从凭经验试错，压缩成一套可以先在小模型上拟合、再在大模型上外推的设计流程。

1.2 历史背景：scaling 不是 LLM 时代突然发明的

课程先回顾理论学习和早期经验研究中的样本复杂度问题。理论文献很早就讨论“样本数增加时误差如何下降”，但那些结果大多是上界；工程上真正需要的是 realized loss，也就是你把模型训出来之后实际能达到的误差。后来的经验工作开始直接拟合数据规模与下游性能或测试损失之间的函数关系，逐步把“规模”从定性直觉变成了可测的曲线。

这段历史回顾的作用非常重要。课程不是想让学生背论文年表，而是要强调：**scaling 这个问题一直都在，只是到了现代神经网络尤其是语言模型时代，这种关系第一次在足够宽的区间里表现得稳定到可以拿来做规划**。



Model	Formula
Exp_3	$y = c - e^{-ax+b}$
Exp_4	$y = c - e^{-ax^{\alpha}+b}$
ExpP_3	$y = c - e^{(x-b)^{\alpha}}$
Pow_3	$y = c - ax^{-\alpha}$
Pow_4	$y = c - (-ax+b)^{-\alpha}$
ILog_2	$y = c - (a/\log x)$

Table 1: Curve families.

图 2: 早期工作已经在比较不同函数族如何拟合“数据量与性能”的关系。

为什么课程要先讲历史

如果不做这一步，学生很容易以为 scaling laws 就是 Kaplan 2020 的几张图。课程真正想建立的是更长的脉络：现代大模型只是把一个老问题推进到了“可以真正拿来指导预算分配”的阶段。

1.3 本章小结

这一节先把第 9 讲的问题定义钉住了：scaling laws 最重要的用途不是给出优美图表，而是帮助我们在真实预算里排除明显错误的训练设计。

2 数据 scaling law：为什么 log-log 图会变直

2.1 什么叫 data scaling law

课程把 data scaling law 定义得很朴素：用一个简单公式，把数据集大小 n 映射到误差或损失。对工程师来说，最重要的不是推导是否完美，而是这条关系在足够大的范围里是否稳定、是否能预测收益递减速度。

一个常见的经验形式是：

$$L(n) \approx L_\infty + An^{-\alpha},$$

其中：

- $L(n)$：在数据规模为 n 时测到的测试损失；
- L_∞：数据无限增大时逼近的极限损失；
- A：决定曲线整体高度的常数；
- α：幂律指数，反映“多加数据到底还有多大收益”。

课程强调，把横轴和纵轴都取对数之后，神经语言模型的损失曲线在一段很长的区间里会接近直线。这一点对工程至关重要，因为直线意味着你只要在较小规模上观察几个点，就有机会估计更大规模的边际收益。

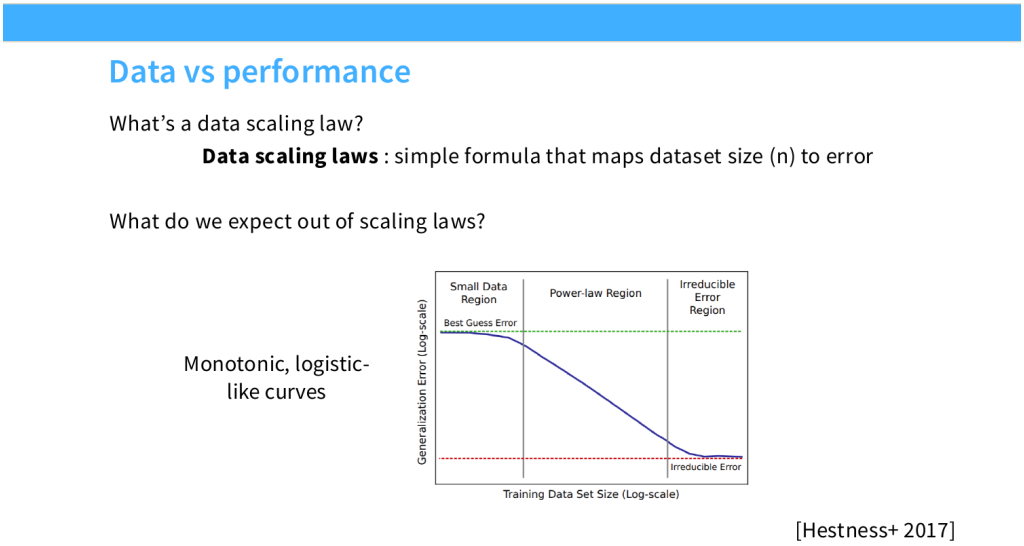


图 3: 课程给出的 data scaling law 直觉图：小数据区、幂律区和不可约误差区。

2.2 均值估计这个玩具例子为什么很关键

课程没有直接把学生扔进复杂模型实验，而是先给出一个极其基础的均值估计例子。这一步很重要，因为它说明幂律并不神秘，它可以直接来自统计估计误差的多项式衰减。

设

$$x_1, \ldots, x_n \sim \mathcal{N}(\mu, \sigma^2), \qquad \hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

则有

$$\mathbb{E}\left[(\hat{\mu} - \mu)^2\right] = \frac{\sigma^2}{n}.$$

取对数可得

$$\log \mathrm{Error} = -\log n + 2\log \sigma.$$

这个例子告诉你：只要误差按某种多项式速度下降，那么 log-log 图上就自然会出现近似直线。课程借它表达的不是“语言模型就像均值估计那么简单”，而是“幂律本身并不离奇，它可能只是复杂学习过程在大尺度下呈现出的统计结构”。

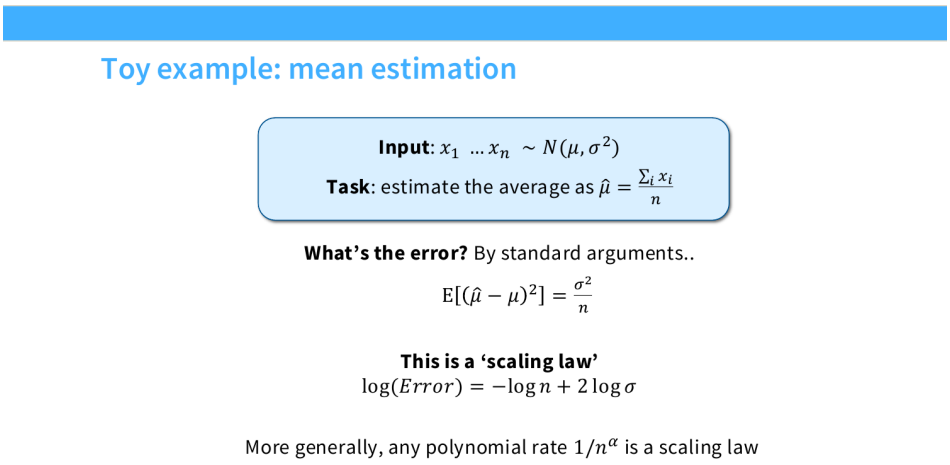


图 4: 均值估计例子展示了最简单的 scaling law：误差随样本数按多项式下降。

玩具例子的真实价值

它把 scaling law 从“经验图表”重新拉回到“统计误差如何衰减”的问题上，帮助我们理解为什么幂律会在实践里反复出现。

2.3 为什么神经网络里的 exponent 仍然是个谜

课程接着指出一个很关键的事实：如果你只拿很多经典模型的直觉，往往会期待接近 $1/n$ 的速度；但在机器翻译、语音和语言建模里，观测到的斜率并不总是如此。这就是所谓“scaling law exponent 的谜团”。

一种解释路径是内在维度理论：虽然原始输入维度很高，但真实数据也许落在低维流形上，因此有效学习难度受内在维度支配。课程对这种解释持谨慎态度，认为它很有启发，但还远谈不上彻底解决。也就是说，**经验规律非常有用，不等于理论已经闭合**。

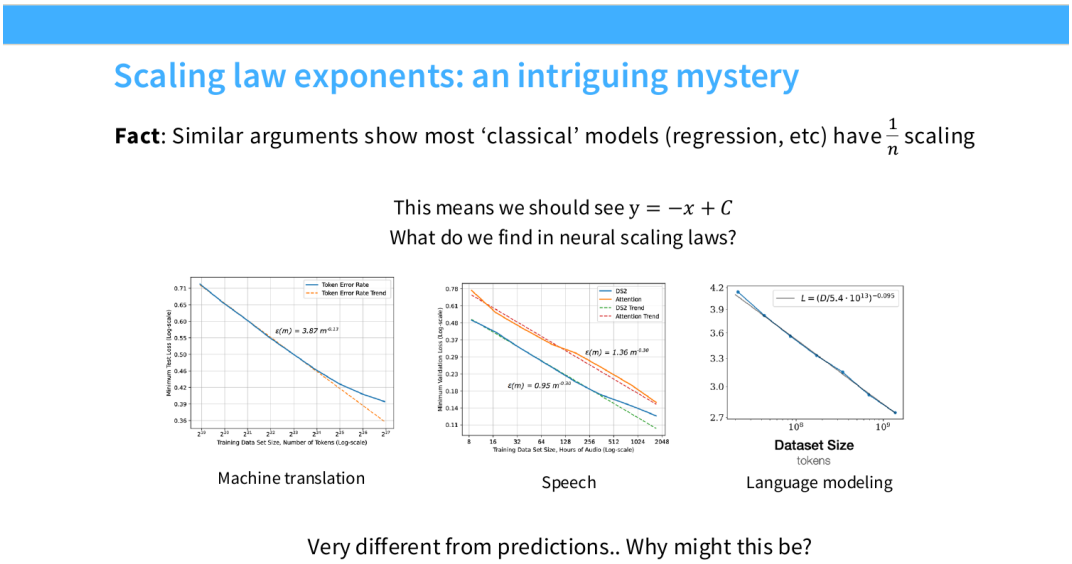


图 5: 课程强调：神经 scaling 的指数与经典统计模型给出的简单预期并不一致。

2.4 本章小结

数据 scaling law 的核心不是“误差会下降”这种空话，而是“在一个足够宽的区间里，下降速度可以被幂律稳定拟合”。课程用均值估计解释幂律为何自然，又提醒学生对指数来源保持理论上的谦逊。

3 数据不只有规模，还有组成、重复和分布偏移

3.1 数据组成往往改的是 offset，而不一定改 slope

第 9 讲后面开始把视角从“有多少数据”扩展到“数据到底是什么”。课程提到，数据组成经常改变的是曲线的 offset，也就是整体高低，而不一定改变幂律斜率。直观地说，更高质量或更贴近目标分布的数据，常常会把整条曲线向下平移，让你在同样 token 数下拿到更低损失；但它不一定改变“规模翻倍之后收益递减”的基本速度。

这一点很重要，因为它把数据工程和 scaling laws 连接起来了。数据混合、筛选和配比不是和 scaling 完全分离的另一块工作，它们其实决定了你最终落在哪条曲线上。

3.2 重复数据为什么不能按名义 token 数来算

课程也讨论了重复数据的代价。在真实世界里，独特数据是有限的；你经常会因为网页镜像、模版文本或近重复内容而反复看到类似样本。问题在于，这些重复 token 的边际价值比全新 token 低得多，因此不能简单按“训练 token 总量”来估收益。

这也是课程提出 effective data 视角的原因。真正值得估算的不是你名义上处理了多少 token，而是其中有多少新的统计信息可以继续推动损失下降。数据重复越高，effective data 与 nominal data 的差距就越大。

关于数据规模最常见的误解

不要把“多训练了很多 token”直接等同于“看到了很多新知识”。如果大量 token 只是重复、模板化改写或近重复文本，那么名义规模会系统性高估真实学习信号。

3.3 本章小结

第 9 讲对数据问题的处理并不粗糙。它明确提醒：规模、组成、重复率和分布偏移都在共同决定曲线形状，工程上真正需要关注的是有效数据而不是生硬的 token 总量。

4 模型工程也可以做 scaling analysis

4.1 除了数据量，还能外推什么

课程回到最初承诺的问题：如果 scaling laws 真能帮我们做设计，那它不该只回答“需要多少数据”，还应该帮助判断架构、优化器、深宽比和 batch 的选择。课程的答案是肯定的。你完全可以在一批较小模型上测试不同架构和训练策略，再根据曲线趋势判断谁在更大规模区间更值得投资。

例如，Transformer 是否会持续优于 LSTM，某个优化器在目标损失附近是否更省算力，深度增加和宽度增加谁的边际收益更好，这些都不必等到最大模型训练完才知道。scaling analysis 的真正力量，就在于把“只能在最贵实验里回答的问题”往前挪。

4.2 critical batch size 说明 batch 不是越大越好

课程还特别讲到 batch size 的 diminishing returns。所谓 critical batch size，可以理解为：在给定目标损失下，batch 扩大到某个点之后，继续变大主要带来并行和吞吐上的好处，但对优化效率本身的帮助迅速减弱。目标损失越低，这个 critical batch 往往越大；但这依然不意味着“batch 能开多大就一定该开多大”。

这条规律和后面系统章节是连着的。batch 不只是优化变量，也是通信、显存和并行策略变量。scaling laws 的价值，在于先告诉你哪里开始收益递减，再决定是否值得为了更大 batch 去承受系统复杂度。

为什么 batch 的 scaling 很重要

如果你不知道最优 batch 随目标损失如何增长，就容易在硬件很贵的情况下把资源浪费在大而无益的全局 batch 上，或者反过来因为 batch 太小而浪费系统吞吐。

4.3 muP 和 scale-aware 学习率只是这一讲的预告

第 9 讲这里只是点到为止，但它已经明确指出一个痛点：如果模型一放大，最优学习率和初始化也跟着漂移，那么“小模型调参再迁移到大模型”这条路就会变得很脆弱。muP 和 scale-aware 初始化/学习率的意义，就是尽可能让这种迁移稳定下来。第 11 讲会把这件事讲得更具体，但第 9 讲已经把问题意识埋下了。

4.4 本章小结

模型工程里的很多选择都可以进入 scaling 框架。课程不是把 scaling laws 限定为几条数据曲线，而是把它扩展成一套可以服务于架构和训练设计的分析工具。

5 联合数据-模型 scaling：回答“该加数据还是加模型”

5.1 真正的训练规划一定是多轴联动的

如果只看数据 scaling，你会问“再加一倍 token 值不值”；如果只看模型 scaling，你会问“再加一倍参数值不值”。但真实训练计划必须同时决定两者，因此课程最后把问题收束成 joint scaling：

$$\mathrm{Error}(N, D) \approx AN^{-\alpha} + BD^{-\beta} + C.$$

这个表达式的意义不是要求你迷信一个统一公式，而是让“模型太小会浪费数据、数据太少会浪费模型”这件事进入定量讨论。

5.2 为什么小模型会浪费大数据

课程强调，很多 token 被“小模型浪费掉”并不是因为数据本身没价值，而是因为模型容量已经成为瓶颈。反过来，模型做得太大但数据不够，也会很快进入数据受限区间。这正是后续 Chinchilla 一类分析能够成立的直觉基础：固定总算力时，模型和数据都要一起长，而且增长比例需要通过经验拟合来确定。

联合 scaling 的工程含义

训练预算不能只沿一个轴堆资源。真正高效的配比，通常来自模型规模和数据规模的同步增长，而不是单边极端扩张。

5.3 本章小结

联合 scaling 把第 9 讲的所有讨论收束成一个真正可操作的问题：给定总预算，模型与数据应该怎样共同放大。后面几讲的大部分现代 recipe，都是在这个主线上继续细化。

6 总结与延伸

第 9 讲最重要的不是单个公式，而是一套训练规划方法。

第一，**scaling laws 的首要用途是预算决策**。课程把它当成一种比大模型盲试更便宜、更系统的预测工具。

第二，**data scaling law 的关键在于幂律区间**。在这个区间里，log-log 图近似线性，因此小规模实验才有外推价值。

第三，**数据问题不止是 token 总数**。组成、质量、重复率和分布偏移都会改变曲线解释，所以有效数据比名义数据更重要。

第四，**模型工程变量同样可以做 scaling analysis**。架构、优化器、batch、深宽比和学习率，不必都等到最大模型上才做判断。

第五，**真正的训练规划是联合规划**。该加模型还是加数据，不能分开问，必须放在同一个预算框架里回答。

我对第 9 讲的压缩提醒

如果只记住“loss 和规模呈幂律”，那只学到了表层。更关键的结论是：**scale 不是盲目放大，而是要让每单位算力尽量落在仍有明显边际收益的区间**。

从课程结构看，第 9 讲是在搭方法论地基；第 11 讲会把这套方法推进到更现代的公开 scaling recipe，包括 muP、WSD 学习率和最近几代开源模型的具体做法。